

一 选择题（每题 0.5 分，共 20 分）

1、二分搜索算法是利用（ A ）实现的算法。

A、分治策略 B、动态规划法 C、贪心法 D、回溯法

2、下列不是动态规划算法基本步骤的是（ B ）。

A、找出最优解的性质 B、构造最优解 C、算出最优解 D、定义最优解

3. 表(23,44,36,48,52,73,64,58)建立哈希表，采用 $h(K)=K\%13$ 计算哈希地址，则元素 64 的哈希地址为(C)。

A. 4 B. 8 C. 12 D. 13

4. 在以下排序算法中，（A、C）是稳定的。

A.合并排序、插入排序 B.快速排序、堆排序

C.Shier（希尔）排序、Select 排序 D.Heap 排序、Merge（归并）排序

算法	平均时间复杂度	最差时间复杂度	空间复杂度	稳定性
插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	是
选择排序	$O(n^2)$	$O(n^2)$	$O(1)$	是（如下实现）
冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$	是
快速排序	$O(n\log n)$	$O(n^2)$	$O(\log n)$	否
堆排序	$O(n\log n)$	$O(n\log n)$	$O(\log n)$	否
归并排序	$O(n\log n)$	$O(n\log n)$	$O(\log n)$	是
希尔排序	$O(n\log n)$	$O(n^2)$	$O(1)$	是（如下实现）

5. 回溯法解旅行售货员问题时的解空间树是（ B ）。

A、子集树 B、排列树 C、深度优先生成树 D、广度优先生成树

6. 下列算法中通常以自底向上的方式求解最优解的是（ B ）。

A、备忘录法 B、动态规划法 C、贪心法 D、回溯法

7、衡量一个算法好坏的标准是（ C ）。

A 运行速度快 B 占用空间少 C 时间复杂度低 D 代码短

8、以下不可以使用分治法求解的是（ D ）。

A 棋盘覆盖问题 B 选择问题 C 归并排序 D 0/1 背包问题

01 背包问题的三种求解方法：

动态规划、回溯法（改造过的深度优先搜索）、分支限界法（改造过的广度优先搜索）

9、下列有关红黑树的描述不正确的是（D）

A. 所有的红链接都为左连接；

B. 一个节点不允许同时连接两个红色链接。

C. 有一个对应的 2-3 树到任何一个红黑树

D. 任何一个空链接到根节点的路径上的黑色链接的数量不一定是相同的。

10. 在无向图的邻接表中，每个顶点的单链表中的节点数等于该顶点的(B)。

A. 入度 B.出度 C. 度 D.度数-1

11. 下面不是分支界限法搜索方式的是（ D ）。

A、广度优先 B、最小耗费优先 C、最大效益优先 D、深度优先

12. 下列算法中通常以深度优先方式系统搜索问题解的是（D ）。

A、备忘录法 B、动态规划法 C、贪心法 D、回溯法

13. 分支限界法和回溯法都是在解空间树 t 上搜索问题的解，它们有（C）

A. 不同的目标但是搜索方法相同

B 不同的搜索方法但是同样的目标

C 不同的目标和不同的搜索方法

D 同样的目标和同样的搜索方法

14. 哈弗曼编码的贪心算法所需的计算时间为（ B ）。

A、 $O(n^2n)$ B、 $O(n\log n)$ C、 $O(2^n)$ D、 $O(n)$

15. 分支限界法解最大团问题时，活结点表的组织形式是（ B ）。

A、最小堆 B、最大堆 C、栈 D、数组

16. 最长公共子序列算法利用的算法是（ B ）。

A、分支界限法 B、动态规划法 C、贪心法 D、回溯法

17.下面是贪心算法的基本要素的是（ C ）。

A、重叠子问题 B、构造最优解 C、贪心选择性质 D、定义最优解

18.回溯法的效率不依赖于下列哪些因素（ D ）

A.满足显约束的值的个数 B. 计算约束函数的时间

C. 计算限界函数的时间 D. 确定解空间的时间

19. （ D ）是贪心算法与动态规划算法的共同点。

A、重叠子问题 B、构造最优解 C、贪心选择性质 D、最优子结构性性质

20、使用分治法求解不需要满足的条件是（ A ）。

A 子问题必须是一样的

B 子问题不能够重复

C 子问题的解可以合并

D 原问题和子问题使用相同的方法解

21、回溯法在问题的解空间树中，按（D）策略，从根结点出发搜索解空间树。

A.广度优先 B. 活结点优先 C.扩展结点优先 D. 深度优先

22、下列算法中不能解决 0/1 背包问题的是（ A ）

A 贪心法 B 动态规划 C 回溯法 D 分支限界法

23. 下列是动态规划算法基本要素的是（ D ）。

A、定义最优解 B、构造最优解 C、算出最优解 D、子问题重叠性质

24. 下列算法中通常以自底向下的方式求解最优解的是（ B ）。

A、分治法 B、动态规划法 C、贪心法 D、回溯法

25. 采用广度优先策略搜索的算法是（ A ）。

A、分支界限法 B、动态规划法 C、贪心法 D、回溯法

26、归并排序算法是利用（ A ）实现的算法。

A、分治策略 B、动态规划法 C、贪心法 D、回溯法

27、背包问题的贪心算法所需的计算时间为（ B ）

A、 $O(n2^n)$ B、 $O(n\log n)$ C、 $O(2^n)$ D、 $O(n)$

28. 0-1 背包问题的回溯算法所需的计算时间为 (A)

A、O (n²n) B、O (nlogn) C、O (2n) D、O (n)

29. 贪心算法与动态规划算法的主要区别是 (B)。

A、最优子结构 B、贪心选择性质 C、构造最优解 D、定义最优解

30. 若要从 1000 个元素中得到 4 个最小值元素，最好采用 (B) 方法。

A. 直接插入排序 B. 直接选择排序 C. 堆排序 D. 快速排序

31. 一个问题可用动态规划算法或贪心算法求解的关键特征是问题的 (B)。

A、重叠子问题 B、最优子结构性性质 C、贪心选择性质 D、定义最优解

32. 以深度优先方式系统搜索问题解的算法称为 (D)。

A、分支界限算法 B、概率算法 C、贪心算法 D、回溯算法

33. 实现最长公共子序列利用的算法是 (B)。

A、分治策略 B、动态规划法 C、贪心法 D、回溯法

34. 算法是由若干条指令组成的有穷序列，而且满足以下性质 (D)

(1) 输入：有 0 个或多个输入

(2) 输出：至少有一个输出

(3) 确定性：指令清晰，无歧义

(4) 有限性：指令执行次数有限，而且执行时间有限

A (1)(2)(3) B(1)(2)(4) C(1)(3)(4) D (1) (2)(3)(4)

35. 函数 $32^n + 10n \log n$ 的渐进表达式是(B).

A. 2^n B. 32^n C. $n \log n$ D. $10n \log n$

36. 回溯法在解空间树 T 上的搜索方式是(A).

A.深度优先 B.广度优先 C.最小耗费优先 D.活结点优先

37. 回溯算法和分支限界法的问题的解空间树不会是(D).

A.有序树 B.子集树 C.排列树 D.无序树

38. 若一个图的边集为{(A,B),(A,C),(B,D),(C,F),(D,E),(D,F)}，则从顶点 A 开始对

该图进行深度优先搜索，得到的顶点序列可能为(B)。

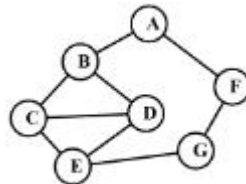
A. A,B,C,F,D,E

B. A,C,F,D,E,B

B. A,B,D,C,F,E

D. A,B,D,F,E,C

39. 若如下图所示的无向连通图，则从顶点 A 开始对该图进行广度优先遍历，得到的顶点序列可能为(D)。



A. A,B,C,D,E,F,G

B. A,B,C,D,E,G,F

C. A,B,F,C,D,E,G

D. A,B,F,C,D,G,E

40. 已知一个有向图的边集为{<a,b>,<a,c>,<a,d>,<b,d>,<b,e>,<d,e>}，则由该图产生的一种可能的拓扑序列为(A)。

A. a,b,c,d,e

B. a,b,d,e,b

C. a,c,b,e,d

D. a,c,d,b,e

二、填空（每空 2 分，共 20 分）

1. 算法的复杂性有_____时间_____复杂性和_____空间_____复杂性之分。

2. 程序是_____算法_____用某种程序设计语言的具体实现。

3. 背包问题的贪心算法

```
void Knapsack(int n, float M, float v[], float w[], float x[])
```

```
{
```

```
    Sort(n, v, w);
```

```
    int i;
```

```
    for (i=1; i<=n; i++) x[i]=0;
```

```
    float c=M;
```

```
    for (i=1; i<=n; i++) {
```

```
        if (w[i]>c) break;
```

```
        x[i]=1;
```

```
        c-=w[i]_____;
```

```

    }
    if (i<=n) x[i]=c/w[i] ;
}

```

4. 快速排序

```

template<class Type>
void QuickSort (Type a[], int p, int r)
{
    if (p<r) {
        int q=Partition(a, p, r);
        QuickSort(a, p, q-1) ; //对左半段排序
        QuickSort(a, q+1, r) ; //对右半段排序
    }
}

```

5、归并排序描述如下：

```

template<class Type>
void Mergesort(Type a[ ], int left, int right)
{
    if (left<right) {
        int i=( left+right )/2;
        Mergesort(a, left, i);
        Mergesort(a,i+1,right);
        Merge(a,b, left, i, right); //合并到数组 b
        Copy(a,b, left, right ); //复制到数组 a
    }
}

```

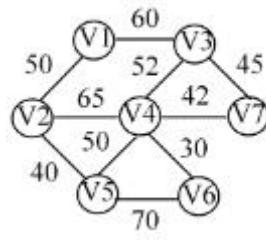
二、 简答题（每题 4 分，共 60 分）

1.算法的五个特点。

有穷性、确定性、可行性、输入和输出

2.已知图所示的一个图，分别描述 Prim 方法和 Kruskal 方法的思想及主要的步骤，

并从顶点 1 出发，给出该图的最小生成树的产生过程。



5. 过程如图6-16所示

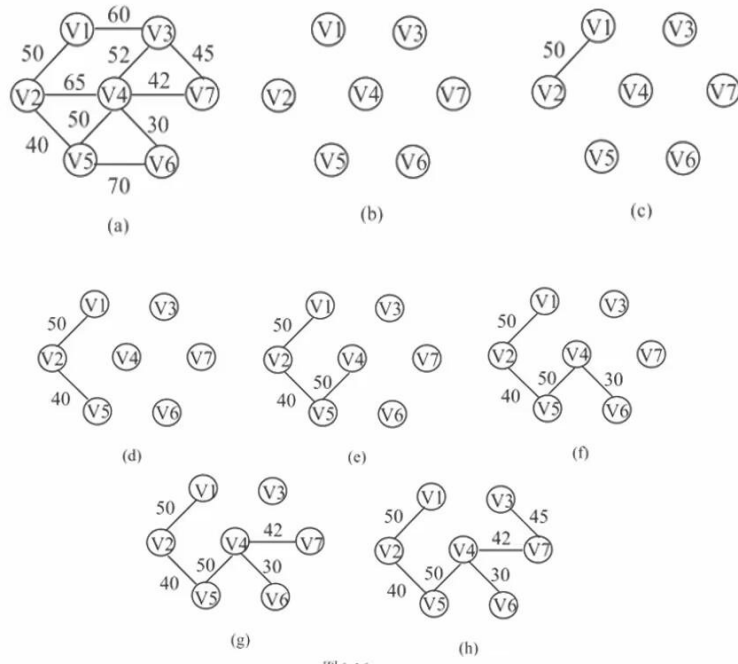


图6-16

6. 求解过程如图6-17所示。

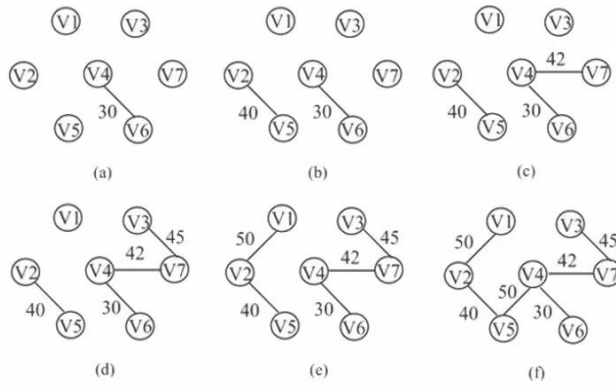
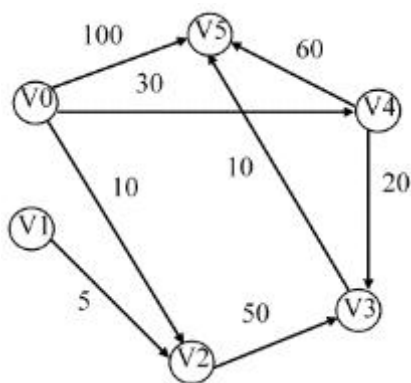


图6-17

3. 下图所示为一个有向图，要求对有向图采用 Dijkstra 算法，求从 V0 到其余各顶点的最短路径。



7. 求解过程如下表所示。

终点	从v0到各终点的D值和最短路径的求解过程
----	----------------------

	i=1	i=2	i=3	i=4	i=5
V1	∞	∞	∞	∞	∞ 无
V2	∞ $\checkmark$ (v_0, v_2)				
V3	∞	∞ $\checkmark$ (v_0, v_4, v_3)	∞ $\checkmark$ (v_0, v_4, v_3)		
V4	∞ $\checkmark$ (v_0, v_4)	∞ $\checkmark$ (v_0, v_4)			
V5	∞ $\checkmark$ (v_0, v_5)	∞ $\checkmark$ (v_0, v_5)	∞ $\checkmark$ (v_0, v_4, v_5)	∞ $\checkmark$ (v_0, v_4, v_3, v_5)	
Vj	V2	V4	V3	V5	
S	{v0, v2}	{v0, v2, v4}	{v0, v2, v3, v4}	{v0, v2, v3, v4, v5}	

4. 描述快速排序的算法思想，并且求一组记录为(46,74,53,14,26,38,86,65, 27,34)进行排序时每一趟的排序结果。

快速排序的算法思想：

快速排序是一种高效的排序算法，其基本思想是通过一趟排序将待排序的记录分割成独立的两部分，其中一部分的所有记录都比另一部分的记录小。然后，对这两部分分别进行快速排序，直到整个序列有序。

具体步骤如下：

1.选择基准元素： 从数组中选择一个元素作为基准，通常选择第一个元素。

2.分割阶段： 将数组中小于基准的元素移到基准的左边，大于基准的元素移到

基准的右边。基准元素在这个过程中找到了其在排序后的最终位置。

3.递归： 递归地对基准左右两侧的子数组进行快速排序。

4.合并： 不需要显式的合并操作，因为在分割阶段就已经确定了基准元素的最终位置。

对记录(46,74,53,14,26,38,86,65,27,34)进行快速排序：

以第一个元素 46 为基准进行排序。每一趟的排序结果如下：

第一趟：

基准：46 排序结果：(34, 26, 38, 14, 27, 46, 74, 65, 53, 86)

第二趟（对基准左侧进行快速排序）：

基准：34 排序结果：(26, 14, 27, 34, 38, 46, 65, 53, 74, 86)

第三趟（对基准右侧进行快速排序）：

基准：65 排序结果：(26, 14, 27, 34, 38, 46, 53, 65, 74, 86)

第四趟（对基准左侧进行快速排序）：

基准：38 排序结果：(26, 14, 27, 34, 38, 46, 53, 65, 74, 86)

第五趟（对基准右侧进行快速排序）：

基准：53 排序结果：(26, 14, 27, 34, 38, 46, 53, 65, 74, 86)

最终的排序结果为：(14, 26, 27, 34, 38, 46, 53, 65, 74, 86)。

5. 描述堆排序的算法思想，并且求一组记录为(46,74,53,14,26,38,86,65,27,34)进行排序时每一趟的排序结果。

堆排序的算法思想：

堆排序是一种基于二叉堆数据结构的排序算法。它利用了最大堆（Max Heap）或最小堆（Min Heap）的性质，在最大堆中，父节点的值总是大于等于其子节点的值；在最小堆中，父节点的值总是小于等于其子节点的值。

堆排序的基本思想是：

1.建堆： 将待排序的数组构建成一个二叉堆。这个过程通常是从数组的最后一个非叶子节点开始，依次向前调整，使得每个节点都满足堆的性质。

2.排序： 将堆顶元素（最大值或最小值，取决于是最大堆还是最小堆）与堆的

最后一个元素交换，然后对除去最后一个元素的部分重新调整成堆，重复这个过程直到整个数组有序。

对记录(46,74,53,14,26,38,86,65,27,34)进行堆排序：

建堆： 从最后一个非叶子节点开始，依次向前调整，得到最大堆。

初始数组： (46,74,53,14,26,38,86,65,27,34)

初始最大堆： (86, 65, 74, 53, 34, 38, 46, 14, 27, 26)

排序： 将堆顶元素（最大值）与堆的最后一个元素交换，然后对除去最后一个元素的部分重新调整成堆。

第一趟排序： (34, 65, 74, 53, 26, 38, 46, 14, 27, 86)

重复排序步骤，直到整个数组有序。

第二趟排序： (38, 53, 74, 65, 26, 27, 46, 14, 34, 86)

第三趟排序： (46, 53, 38, 65, 26, 27, 14, 74, 34, 86)

第四趟排序： (34, 46, 38, 27, 26, 53, 14, 65, 74, 86)

第五趟排序： (26, 46, 34, 27, 14, 38, 53, 65, 74, 86)

第六趟排序： (38, 46, 34, 27, 14, 26, 53, 65, 74, 86)

最终的排序结果为： (14, 26, 27, 34, 38, 46, 53, 65, 74, 86)。

6. 描述分治法的算法思想，并且求一组记录为(46,74,53,14,26,38,86,65, 27,34)进行归并排序时每一趟的排序结果。

分治法的算法思想：

分治法是一种算法设计策略，它将问题分解成若干个规模较小、结构与原问题相似的子问题，然后递归地解决这些子问题，最后将这些子问题的解合并起来，得到原问题的解。

该算法思想通常包含以下步骤：

1.分解（Divide）： 将原问题分解成若干个规模较小的子问题，这些子问题是原问题的规模较小的实例。

2.求解（Conquer）： 递归地解决这些子问题，如果子问题足够小，则直接求解。

3.合并（Combine）： 将子问题的解合并成原问题的解。

归并排序的思想：

归并排序是使用分治法解决排序问题的经典算法。其基本思想是将待排序的数组分成两部分，分别对这两部分进行排序，然后将排好序的子数组合并成一个有序数组。

对记录(46,74,53,14,26,38,86,65,27,34)进行归并排序：

分解： 将数组分解成单个元素或者空数组（递归到最小子问题）。

合并排序： 将子问题的解按顺序合并成一个有序数组。

下面是每一趟归并排序的结果：

初始数组： (46,74,53,14,26,38,86,65,27,34)

第 1 趟归并： (46,74) 和 (53,14) 合并排序 ->(14,46,53,74)

第 2 趟归并： (26,38) 和 (65,27) 合并排序 ->(26,27,38,65)

第 3 趟归并： (14,46,53,74) 和 (26,27,38,65) 合并排序 ->(14,26,27,38,46,53,65,74)

第 4 趟归并： (86) 和 (34) 合并排序 ->(34,86)

第 5 趟归并： (34,86) 和 (14,26,27,38,46,53,65,74) 合并排序 ->(14,26,27,34,38,46,53,65,74,86)

最终的排序结果为： (14,26,27,34,38,46,53,65,74,86)。

7. 描述平衡查找算法的基本思想，并说明红黑树和 2-3 树之间的关系。

平衡查找算法的基本思想：

平衡查找算法的目标是保持查找树的平衡，以确保各种操作（插入、删除、查找等）的时间复杂度能够保持在较低的水平。平衡查找树的基本思想是通过在插入或删除节点时对树进行一系列的旋转和调整操作，使得树的结构保持平衡，即保持树的高度较低。

红黑树和 2-3 树的关系：

红黑树是一种自平衡的二叉查找树，它具有以下特性：

- 1.每个节点要么是红色，要么是黑色。
- 2.根节点是黑色的。

3.每个叶子节点（NIL 节点，空节点）是黑色的。

4.如果一个节点是红色的，那么它的两个子节点都是黑色的。

5.从任意节点到其每个叶子的所有路径都包含相同数目的黑色节点。

红黑树的结构和 2-3 树之间存在一一对应的关系，即任何红黑树都可以转换为一个等价的 2-3 树，反之亦然。这种对应关系可以通过以下对应规则描述：

6.红黑树的黑色节点对应 2-3 树中的 2 节点。

7.红黑树的红色节点对应 2-3 树中的 3 节点。

通过这种对应关系，红黑树的插入、删除等操作可以看作是对应 2-3 树的操作。在实际应用中，红黑树更加简单，而且由于其结构的对应关系，操作的实现也更为直观。这使得红黑树成为了一种广泛应用的平衡查找树的实现。

8. 写出插入将一组键值为 42 37 12 18 6 11 48 50 88 66 插入红黑树的过程，并写出其对应的 2-3 树。

在红黑树中插入节点的过程涉及到颜色的变化和树的旋转。下面是将一组键值为 42, 37, 12, 18, 6, 11, 48, 50, 88, 66 插入红黑树的过程，并附带对应的 2-3 树。

1.插入 42:

2.插入后红黑树: (42)

3.对应 2-3 树: [42]

4.插入 37:

5.插入后红黑树: (37, 42)

6.对应 2-3 树: [37, 42]

7.插入 12:

8.插入后红黑树: (37, 12, 42)

9.对应 2-3 树: [12, 37, 42]

10.插入 18:

11.插入后红黑树: (37, 12, 18, 42)

12.对应 2-3 树: [12, 18, 37, 42]

13.插入 6:

14.插入后红黑树：（18, 6, 37, 12, 42）

15.对应 2-3 树：[6, 12, 18, 37, 42]

16.插入 11:

17.插入后红黑树：（18, 6, 12, 11, 37, 42）

18.对应 2-3 树：[6, 11, 12, 18, 37, 42]

19.插入 48:

20.插入后红黑树：（18, 6, 12, 11, 37, 42, 48）

21.对应 2-3 树：[6, 11, 12, 18] 和 [37, 42, 48]

22.插入 50:

23.插入后红黑树：（18, 6, 12, 11, 37, 42, 48, 50）

24.对应 2-3 树：[6, 11, 12, 18] 和 [37, 42, 48, 50]

25.插入 88:

26.插入后红黑树：（37, 18, 6, 12, 11, 42, 48, 50, 88）

27.对应 2-3 树：[6, 11, 12, 18] 和 [37, 42, 48, 50, 88]

28.插入 66:

29.插入后红黑树：（37, 18, 6, 12, 11, 42, 48, 50, 88, 66）

30.对应 2-3 树：[6, 11, 12, 18] 和 [37, 42, 48, 50] 和 [66, 88]

最终的红黑树和对应的 2-3 树都是平衡的，符合红黑树和 2-3 树的性质。请注意，红黑树的具体结构可能因为旋转操作的不同而有所变化，但最终满足红黑树的所有性质。这里的插入过程是一种可能的示范。

9. 写出插入将一组键值为 42 37 12 18 6 11 48 50 88 66 插入 2-3 树的过程，并写出其对应的红黑树。

在 2-3 树中插入节点的过程较为复杂，因为 2-3 树可以是 2 节点或 3 节点。下面是将一组键值为 42, 37, 12, 18, 6, 11, 48, 50, 88, 66 插入 2-3 树的过程，并伴随对应的红黑树。

1.插入 42:

2.插入后 2-3 树：[42]

3.对应红黑树：（42）

4.插入 37：

5.插入后 2-3 树：[37, 42]

6.对应红黑树：（37， 42）

7.插入 12：

8.插入后 2-3 树：[12, 37, 42]

9.对应红黑树：（37， 12， 42）

10.插入 18：

11.插入后 2-3 树：[12, 18, 37, 42]

12.对应红黑树：（37， 18， 12， 42）

13.插入 6：

14.插入后 2-3 树：[6, 12, 18, 37, 42]

15.对应红黑树：（18， 6， 37， 12， 42）

16.插入 11：

17.插入后 2-3 树：[6, 11, 12, 18, 37, 42]

18.对应红黑树：（18， 6， 12， 11， 37， 42）

19.插入 48：

20.插入后 2-3 树：[6, 11, 12, 18] 和 [37, 42, 48]

21.对应红黑树：（18， 6， 12， 11， 37， 42， 48）

22.插入 50：

23.插入后 2-3 树：[6, 11, 12, 18] 和 [37, 42, 48, 50]

24.对应红黑树：（37， 18， 6， 12， 11， 42， 48， 50）

25.插入 88：

26.插入后 2-3 树：[6, 11, 12, 18] 和 [37, 42, 48, 50, 88]

27.对应红黑树：（37， 18， 6， 12， 11， 42， 48， 50， 88）

28.插入 66:

29.插入后 2-3 树: [6, 11, 12, 18] 和 [37, 42, 48, 50] 和 [66, 88]

30.对应红黑树: (37, 18, 6, 12, 11, 42, 48, 50, 88, 66)

最终的 2-3 树和对应的红黑树都是平衡的, 符合 2-3 树和红黑树的性质。请注意, 具体的插入过程可能会有多种变化, 但最终的树结构应满足 2-3 树和红黑树的性质。这里的插入过程是一种可能的示范。

10. 将键 EASYQUTION 依次插入一张初始为空且大小为 $M=5$ 的基于线性探测法的散列表中。(使用散列函数 $11k \% M$ 将第 k 个字母散列到某个数组索引上)

首先, 我们使用散列函数 ($h(k) = 11 \times k \bmod M$) 将每个键散列到散列表的某个索引上, 其中 ($M = 5$)。然后, 我们使用线性探测法解决冲突, 即如果散列到的位置已经被占用, 就向后线性地查找下一个可用的位置。

让我们按顺序插入键 E, A, S, Y, Q, U, T, I, O, N 到大小为 ($M = 5$) 的散列表中。

1.插入键 E:

2.($h(E) = 11 \times 5 \bmod 5 = 0$), 位置 0 空, 插入 E 到位置 0。

散列表状态: [E, , , ,]

3.插入键 A:

4.($h(A) = 11 \times 1 \bmod 5 = 1$), 位置 1 空, 插入 A 到位置 1。

散列表状态: [E, A, , ,]

5.插入键 S:

6.($h(S) = 11 \times 19 \bmod 5 = 4$), 位置 4 空, 插入 S 到位置 4。

散列表状态: [E, A, , , S]

7.插入键 Y:

8.($h(Y) = 11 \times 25 \bmod 5 = 0$), 位置 0 已占用, 线性探测找到下一个空位置, 即位置 1, 插入 Y 到位置 1。

散列表状态: [E, A, Y, , S]

9.插入键 Q:

10. ($h(Q) = 11 \times 17 \bmod 5 = 1$), 位置 1 已占用, 线性探测找到下一个空位置, 即位置 2, 插入 Q 到位置 2。

散列表状态: [E, A, Y, Q, S]

11. 插入键 U:

12. ($h(U) = 11 \times 21 \bmod 5 = 0$), 位置 0 已占用, 线性探测找到下一个空位置, 即位置 3, 插入 U 到位置 3。

散列表状态: [E, A, Y, Q, S, U]

13. 插入键 T:

14. ($h(T) = 11 \times 20 \bmod 5 = 0$), 位置 0 已占用, 线性探测找到下一个空位置, 即位置 4, 插入 T 到位置 4。

散列表状态: [E, A, Y, Q, S, U, T]

15. 插入键 I:

16. ($h(I) = 11 \times 9 \bmod 5 = 2$), 位置 2 已占用, 线性探测找到下一个空位置, 即位置 3, 插入 I 到位置 3。

散列表状态: [E, A, Y, Q, S, U, T, I]

17. 插入键 O:

18. ($h(O) = 11 \times 15 \bmod 5 = 2$), 位置 2 已占用, 线性探测找到下一个空位置, 即位置 4, 插入 O 到位置 4。

散列表状态: [E, A, Y, Q, S, U, T, I, O]

19. 插入键 N:

20. ($h(N) = 11 \times 14 \bmod 5 = 3$), 位置 3 已占用, 线性探测找到下一个空位置, 即位置 4, 插入 N 到位置 4。

散列表状态: [E, A, Y, Q, S, U, T, I, O, N]

现在, 我们成功地将所有键插入了基于线性探测法的散列表中。

11. 分别对 0-1 背包问题和最短路径问题, 描述贪心法的求解思想, 并写出其主要步骤, 并分析其复杂度.

0-1 背包问题的贪心法求解思想: 0-1 背包问题是一个经典的组合优化问题, 贪心法通常不适用于该问题, 因为它要求做出的选择是整数, 而贪心法通常会选择

局部最优解，但不一定能够得到全局最优解。对于 0-1 背包问题，动态规划是更为常用的解决方法。

最短路径问题的贪心法求解思想：

最短路径问题是寻找两个节点之间路径长度最短的问题。贪心法在解决最短路径问题时，常用的算法是 Dijkstra 算法。Dijkstra 算法的基本思想是从起始节点开始，每次选择当前最短路径的节点作为下一个访问节点，然后更新与该节点相邻的节点的最短路径。贪心地选择当前最短路径的节点，直到到达目标节点为止。

Dijkstra 算法的主要步骤：

- 1.初始化：将起始节点的最短路径设置为 0，其他节点的最短路径设置为无穷大。
- 2.从起始节点开始，选择当前最短路径的节点，标记为已访问。
- 3.更新与当前节点相邻的节点的最短路径，如果通过当前节点到达相邻节点的路径比原先记录的路径短。
- 4.重复步骤 2 和步骤 3，直到所有节点都被访问。
- 5.最终得到起始节点到每个节点的最短路径。

Dijkstra 算法的复杂度分析：

- 6.对每个节点的访问和更新最短路径的操作都需要 $O(V)$ 的时间，其中 V 是节点的数量。
- 7.如果使用优先队列来快速找到当前最短路径的节点，那么总体的时间复杂度为 $O((V + E)\log V)$ ，其中 E 是边的数量。
- 8.空间复杂度主要取决于存储最短路径和节点访问状态的数据结构，通常为 $O(V)$ 。

总体而言，Dijkstra 算法是解决单源最短路径问题的一种高效算法，但要注意其对于带有负权边的图可能不适用，因为它要求所有边的权重都为非负数。

12. 分别对 0-1 背包问题和最短路径问题，描述动态规划法的求解思想，并写出其主要步骤，并分析其复杂度.

0-1 背包问题的动态规划求解思想：

0-1 背包问题是一个经典的组合优化问题，动态规划是一种常用于解决此类问题的方法。动态规划的基本思想是将原问题分解为子问题，并通过解决子问题来构建原问题的解。对于 0-1 背包问题，可以使用动态规划来寻找最优解。

动态规划的主要步骤：

1.定义状态：定义子问题中的状态。在 0-1 背包问题中，可以定义状态为 $dp[i][j]$ ，表示在前 i 个物品中，背包容量为 j 时的最大价值。

2.状态转移方程：确定状态之间的转移关系。对于 0-1 背包问题，状态转移方程可以表示为：

$$dp[i][j] = \max(dp[i-1][j], dp[i-1][j-w[i]] + v[i])$$

其中， $w[i]$ 是第 i 个物品的重量， $v[i]$ 是第 i 个物品的价值。这个方程表示在考虑第 i 个物品时，可以选择放入背包，也可以选择不放入背包。

3.初始化：初始化边界条件，例如 $dp[0][j]$ 和 $dp[i][0]$ 的值。

4.递推计算：使用状态转移方程递推计算出 $dp[i][j]$ 的值，从小规模的子问题逐步构建出原问题的解。

5.结果提取：最终结果通常存储在 dp 数组的右下角，即 $dp[N][C]$ ，其中 N 是物品的数量， C 是背包的容量。

动态规划的复杂度分析：

6.时间复杂度：在计算 dp 数组的过程中，需要填充每个元素，每个元素的计算涉及常数次的操作。因此，动态规划的时间复杂度通常为 $O(NC)$ ，其中 N 是物品的数量， C 是背包的容量。

7.空间复杂度：需要维护一个二维的 dp 数组，因此空间复杂度为 $O(NC)$ 。

最短路径问题的动态规划求解思想：

动态规划也可用于解决最短路径问题，其中 Floyd-Warshall 算法是一种经典的动态规划算法。

动态规划的主要步骤：

8.定义状态：定义状态为 $dp[i][j][k]$ ，表示从节点 i 到节点 j 的路径中，经过的中间节点集合为 $\{k\}$ 的最短路径长度。

9.状态转移方程：确定状态之间的转移关系。Floyd-Warshall 算法的状态转移方程为：

$$dp[i][j][k] = \min(dp[i][j][k], dp[i][k][k] + dp[k][j][k])$$

其中， $dp[i][j][k]$ 表示经过节点 k 的最短路径长度。

10.初始化： 初始化 $dp[i][j][k]$ ，其中 $dp[i][j][0]$ 等于从节点 i 到节点 j 的直接路径长度。

11.递推计算： 使用状态转移方程递推计算 $dp[i][j][k]$ 的值。

12.结果提取： 最终结果通常存储在 dp 数组中，表示从起始节点到目标节点的最短路径长度。

动态规划的复杂度分析：

13.时间复杂度：Floyd-Warshall 算法的时间复杂度为 $O(V^3)$ ，其中 V 是节点的数量。

14.空间复杂度：需要维护一个三维的 dp 数组，因此空间复杂度为 $O(V^3)$ 。

总体而言，动态规划是一种强大的问题求解方法，但在某些情况下可能因状态空间过大而导致计算复杂度较高。

13. 对于数塔问题，描述动态规划的求解思想，并写出其主要步骤，并分析其复杂度.

数塔问题的动态规划求解思想：

数塔问题是一个经典的动态规划问题，通常描述为一个由数字组成的三角形，从顶部开始向底部移动，每一步只能移动到下一行相邻的数字。问题的目标是找到一条从顶部到底部的路径，使得路径上的数字之和最大。动态规划的思想可以用来有效地解决这个问题。

动态规划的主要步骤：

1.定义状态： 定义子问题中的状态。在数塔问题中，可以定义状态为 $dp[i][j]$ ，表示从顶部到达数塔第 i 行第 j 列的最大路径和。

2.状态转移方程： 确定状态之间的转移关系。对于数塔问题，状态转移方程可以表示为：

$$dp[i][j] = tower[i][j] + \max(dp[i+1][j], dp[i+1][j+1])$$

其中， $tower[i][j]$ 是数塔第 i 行第 j 列的数字， $\max(dp[i+1][j], dp[i+1][j+1])$ 表示从下一行移动到相邻的数字中选择最大的路径和。

3.初始化： 初始化边界条件，即最底层的 dp 数组，它等于数塔底部相应位置的数字。

4.递推计算： 使用状态转移方程递推计算出 $dp[i][j]$ 的值，从底部逐步向上构建

出最优解。

5.结果提取： 最终结果通常存储在 `dp` 数组的顶部，即 `dp[0][0]`。

动态规划的复杂度分析：

6.时间复杂度： 计算 `dp` 数组的过程中，每个元素的计算涉及常数次的操作，因此时间复杂度为 $O(N^2)$ ，其中 N 是数塔的行数。

7.空间复杂度： 需要维护一个二维的 `dp` 数组，因此空间复杂度为 $O(N^2)$ 。

总体而言，动态规划是解决数塔问题的有效方法，通过从底部向上的动态规划计算，可以避免重复计算，提高效率。

14. 对 0-1 背包问题，描述回溯法的求解思想，并写出其主要步骤，并分析其复杂度.

0-1 背包问题的回溯法求解思想：

0-1 背包问题是一个经典的组合优化问题，要求在限定总重量的情况下，选择不同物品放入背包，使得背包中物品的总价值最大。回溯法是一种通过不断尝试所有可能的组合来求解问题的方法。

回溯法的主要步骤：

1.定义问题的状态： 在 0-1 背包问题中，状态可以定义为当前考虑的物品和当前已经装入背包的重量。

2.确定递归结束条件： 当考虑完所有物品或者背包达到了限定重量时，递归结束。

3.确定选择： 对于每个物品，可以选择放入背包或不放入背包，这是一个二叉决策。

4.递归尝试所有可能的组合： 通过递归调用，在每一步选择后继续尝试下一个物品，直到达到递归结束条件。

5.回溯： 在递归过程中，如果发现当前组合不满足问题的要求（比如超过了背包的重量限制），则需要回溯到上一步，取消之前的选择，尝试其他可能性。

6.记录最优解： 在递归的过程中，可以记录当前已经得到的最大价值，以便在整个搜索过程结束后找到最优解。

回溯法的复杂度分析：

7.时间复杂度： 回溯法会尝试所有可能的组合，因此其时间复杂度通常较高，

是指数级别的，即 $O(2^n)$ ，其中 n 是物品的数量。

8.空间复杂度：回溯法的空间复杂度主要取决于递归调用的深度，通常是 $O(n)$ ，其中 n 是物品的数量。这是因为递归调用过程中需要维护一个递归栈。

总体而言，回溯法是一种直观且简单的求解组合优化问题的方法，但其指数级别的时间复杂度使得在问题规模较大时效率较低。在实际应用中，通常需要结合一些剪枝策略，如动态规划思想，来提高算法的效率。

15.对 0-1 背包问题，描述分支限界法的求解思想，并写出其主要步骤，并分析其复杂度.

0-1 背包问题的分支限界法求解思想：

分支限界法是一种优化的搜索算法，它在解空间中通过剪枝操作，去除一些明显不可能产生最优解的分支，从而减少搜索空间。对于 0-1 背包问题，分支限界法基于贪心策略和优先队列进行搜索，以更有效地寻找最优解。

分支限界法的主要步骤：

1.初始化：将问题转化为一个树形结构的搜索空间，初始节点对应于问题的初始状态。

2.使用优先队列：将节点按照某种优先级（通常是节点的价值上界或者价值下界）放入优先队列中。

3.循环：从优先队列中选择一个节点进行扩展，即生成该节点的子节点。对于每个子节点，计算其价值上界，将其加入优先队列。

4.剪枝操作：如果某个节点的价值上界小于当前已知的最优解，可以剪枝，不再对该节点进行扩展。

5.更新最优解：如果扩展的节点是叶子节点（无法再扩展），更新当前已知的最优解。

6.重复：重复上述步骤，直到找到最优解或者搜索空间为空。

分支限界法的复杂度分析：

7.时间复杂度：分支限界法的时间复杂度通常较低，因为通过剪枝操作，减少了搜索空间。在最坏情况下，其复杂度为 $O(b^d)$ ，其中 b 是每个节点的分支数， d 是搜索树的深度。

8.空间复杂度：分支限界法的空间复杂度主要取决于优先队列的大小，通常是

$O(b^d)$ 。在实际应用中，可以通过一些优化策略，如状态压缩等，减少空间复杂度。

总体而言，分支限界法在解决 0-1 背包问题等组合优化问题时，通过巧妙的剪枝操作，能够在搜索空间较大的情况下高效地找到最优解。